

Recursive Inequality Logic for SPY Options Using Combinatorics and Group Theory

Technical Research Report: Mathematical and Computational Strategies for Recursive Logic Engines in SPY Options Trading with Mamba2 and Neural CDEs

Introduction

The landscape of algorithmic options trading, particularly for highly liquid instruments like SPY (S&P 500 ETF) options, is evolving rapidly with the integration of advanced machine learning architectures and mathematical logic frameworks. The challenge addressed in this report is the design and implementation of a recursive logic engine capable of encoding, evaluating, and optimizing highly complex, nested inequality structures as predicates within a hybrid neural-symbolic system. This engine must support:

- Inequality sets with up to 200+ chained inequalities (e.g.,

$$(H[3] - H[2] > H[5] - H[3] < H[10] - C[8] \dots)$$

), with up to four such sets combined via AND/OR.

- Hierarchical comparison of these sets (up to four super sets), where sets themselves are compared using

$$< , > , =$$

- Integration with advanced models: Mamba2 (state-space models), Neural Controlled Differential Equations (Neural CDEs), seven diffusion-based forward market predictors, 14 institutional trading rules, and a fuzzy logic-based money management system.
- Efficient handling of a rich feature set: 52 columns including OHLCV, Greeks (delta, theta), implied volatility metrics (IV, IVR), technical indicators (RSI_{dynamic}, PSAR_{dynamic}, BB_{dynamic}), and proprietary features (upward/downward pressure, etc.).

This report systematically explores the mathematical formalism, combinatorial and algebraic strategies, computational graph representations, CUDA-accelerated implementation, and Python code patterns necessary to realize such an engine. The focus is on integrating combinatorics, set theory, and group theory into the model training process, enabling efficient reasoning over recursive, nested logical structures.

1. Mathematical Formalism for Recursive Inequality Structures

1.1. Notation and Predicate Structure

Let

$$\mathcal{F} = \{f_1, f_2, \dots, f_{52}\}$$

denote the set of features (columns) per time step. For each time index t , define

$$H[t], L[t], C[t], O[t], V[t]$$

(high, low, close, open, volume), and so on, as elements of $\mathcal{F}$.

A chained inequality is a sequence of expressions of the form:

$$E_1 \circ_1 E_2 \circ_2 E_3 \cdots \circ_{n-1} E_n$$

where each E_i is an arithmetic expression over features at various time indices, and

$$\circ_i \in \{<, >, =\}$$

An inequality set S is a conjunction (AND) or disjunction (OR) of up to 200 such chained inequalities:

$$S = \bigwedge_{j=1}^{200} I_j \text{ or } S = \bigvee_{j=1}^{200} I_j$$

where each I_j is a chained inequality.

A super set $\mathcal{S}$ is a higher-level structure:

$$\mathcal{S} = (S_1 \# S_2 \# S_3 \# S_4)$$

with up to four sets S_i , each compared using

$$<, >, =$$

and possibly combined recursively.

1.2. Variational and Hierarchical Inequalities

The recursive logic engine must support hierarchical variational inequalities—a mathematical framework where solutions to one inequality define the feasible set for another. In formal terms, given operators F_1, F_2 and convex sets K_1, K_2 , the hierarchical variational inequality problem is:

$$\text{Find } x^* \in K_2 \text{ such that } \langle F_1(x^*), x - x^* \rangle \geq 0, \forall x \in K_2,$$

where

$$K_2 = \{x \mid \langle F_2(x), y - x \rangle \geq 0, \forall y \in K_1\}$$

This formalism naturally encodes nested logical constraints and is amenable to operator splitting and fixed-point iterative methods for efficient solution.

1.3. Set and Group Theoretic Structure

Each set of inequalities can be viewed as a totally ordered set or a coarse-grained partition over the space of possible feature values. Group theory provides a framework for identifying symmetries and invariants in the structure of these sets, which can be exploited for computational efficiency and model generalization.

2. Combinatorics for Efficient Exploration of Inequality Permutations

2.1. Permutations and Combinations in Predicate Generation

Given the vast number of possible chained inequalities (e.g., selecting 3 out of 52 features for each expression, choosing operators, and time indices), the combinatorial explosion is significant. The number of possible permutations for n features is:

$$n!$$

and the number of combinations for selecting r features from n is:

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

For 52 features and 200 chained inequalities, the number of possible unique sets is astronomical. Efficient exploration requires:

- **Pruning:** Use domain knowledge (e.g., only allow certain feature combinations, restrict time lags) to reduce the search space.
- **Sampling:** Employ random or stratified sampling to select representative subsets for evaluation.
- **Beam Search:** Use heuristic search algorithms (e.g., beam search) to maintain a tractable set of the most promising predicate chains at each level.

2.2. Combinatorial Optimization and Search Heuristics

Beam search is particularly effective for exploring large predicate graphs. At each level, only the top k partial solutions (based on a heuristic score, e.g., predictive power or information gain) are retained and expanded. This approach balances exploration and computational tractability, and can be parallelized for GPU acceleration.

Iterative beam search and bi-directional search can further enhance efficiency, especially when the predicate graph has a hierarchical or recursive structure. These methods have been shown to find near-optimal solutions in large combinatorial spaces with reasonable computational resources.

3. Set Theory and Group Theory for Structuring and Evaluating Nested Inequality Sets

3.1. Set Theory: Coarse-Grained Partitions and Set Inequalities

Set theory provides the language for expressing inclusion, intersection, and ordering relationships among sets of inequalities. Key concepts include:

- **Subset and superset relations:**

$$S_1 \subseteq S_2$$

- **Set inequalities:**

$$S_1 < S_2$$

- **Coarse-grained partitions:** Partitioning the space of possible predicate evaluations into equivalence classes enables efficient aggregation and density estimation.

Table 1: Types of Set Inequality

Notation	Meaning
$A \subseteq B$	A is a subset of B
$A \supseteq B$	A is a superset of B
$A \subset B$	A is a proper subset of B
$A \supset B$	A is a proper superset of B
$A < B$	All elements of A are less than B

3.2. Group Theory: Symmetry, Invariants, and Permutation Groups

Group theory enables the identification and exploitation of symmetries in the predicate graph:

- **Symmetry operations:** Permutations of feature indices or time lags that leave the logical structure invariant.
- **Group actions:** The set of all permutations forms a group under composition.
- **Invariants:** Quantities unchanged under group actions.

Example: Symmetric Group S_n

The symmetric group:

$$S_n$$

consists of all permutations of n elements. In the context of chained inequalities, this group captures all possible reorderings of feature indices. By considering only unique orbits under S_n , the number of distinct predicate structures can be dramatically reduced.

4. Encoding Recursive Nested Inequalities as Graph Predicates

4.1. Logic Graph Representation

The recursive structure of nested inequalities is naturally represented as a directed acyclic graph (DAG), where:

- Nodes represent atomic predicates (e.g.,

$$H[3] - H[2] > H[5] - H[3]$$

), logical operators (AND, OR), or set comparisons.

- Edges encode the logical flow and dependencies between predicates and their aggregations.

This graph representation supports:

- **Efficient evaluation:** Each node can cache its result, enabling memoization and parallel evaluation.
- **Differentiable logic:** By relaxing logical operators to differentiable functions (e.g., using t-norms for AND/OR), the entire graph can be integrated into neural network training pipelines.

Figure 1: Example Predicate Graph (Logical Structure)

$$\begin{aligned} &[H[3] - H[2] > H[5] - H[3]] \rightarrow \text{AND} \rightarrow [\text{Set 1}] \\ &[H[10] - C[8] < \dots] \rightarrow \\ &[\dots 200 + \text{inequalities } \dots] \rightarrow (\text{AND/OR}) \rightarrow [\text{Super Set}] \end{aligned}$$

4.2. Efficient Data Structures

Given the scale (200+ inequalities per set, 4 sets per super set, up to 4 super sets), sparse representations are essential:

- **Sparse matrices and tensors:** Store only non-zero or active predicates, reducing memory footprint.
- **Compressed representations:** Use delta encoding, run-length encoding, or dictionary coding for time-series features to further reduce storage and computation costs.

5. Integration with Mamba2 and Neural CDEs: CUDA-Accelerated Training Pipelines

5.1. Mamba2 Architecture and CUDA Integration

Mamba2 is a state-of-the-art state-space model architecture optimized for sequence modeling. It supports CUDA acceleration and is compatible with PyTorch 2.x. Key implementation steps include:

- **Environment setup:** Use Python 3.10, CUDA 11.8+, PyTorch 2.1.1+, and install `mamba_ssm` and `causal_conv1d` as per official instructions.
- **Model instantiation:** Mamba2 models can be initialized with custom dimensions and deployed on GPU for efficient training and inference.

5.2. Neural Controlled Differential Equations (Neural CDEs)

Neural CDEs generalize RNNs to continuous time and are particularly well-suited for modeling time-series with irregular sampling and complex dependencies. They can be integrated with logical predicate graphs by:

- **Encoding predicate evaluations as control signals:** The output of the logic graph at each time step can serve as an input channel to the CDE.
- **Differentiable logic:** By using fuzzy logic or t-norm relaxations, the predicate graph remains differentiable, allowing gradients to flow through the entire system during training.

5.3. CUDA-Accelerated Custom Operators

For maximum efficiency, especially when evaluating large numbers of predicates in parallel, custom CUDA kernels can be implemented:

- **Arithmetic filters and interval arithmetic:** Use interval arithmetic to quickly evaluate predicates with floating-point uncertainty, falling back to exact arithmetic only when necessary.
- **Batch evaluation:** Design kernels that process batches of predicate evaluations, leveraging the SIMD architecture of GPUs.
- **Integration with PyTorch:** Use PyBind11 or custom autograd functions to expose CUDA kernels as PyTorch operations, enabling seamless integration with neural network training pipelines.

6. Optimization and Training Strategies with Logical Constraints

6.1. Integrating Logical Constraints into Model Training

Logical constraints can be incorporated into neural network training via:

- **Penalty methods:** Add a penalty term to the loss function proportional to the degree of constraint violation.
- **Lagrange multipliers:** Introduce dual variables to enforce hard constraints during optimization.
- **Semantic loss:** Use a differentiable loss function that measures the probability of satisfying logical constraints, enabling end-to-end training.

Example: Semantic Loss for Exactly-One Constraint

$$\mathcal{L}_s(\text{exactly_one}, p) \propto \log \left(\sum_{i=1}^n p_i \prod_{\substack{j=1 \\ j \neq i}}^n (1 - p_j) \right)$$

where p_i are the predicted probabilities for each output.

6.2. Differentiable Fuzzy Logic and Symbolic-Neural Integration

Fuzzy logic provides a smooth relaxation of Boolean logic, enabling gradient-based optimization:

- **T-norms:** Use product, Łukasiewicz, or Zadeh t-norms to implement differentiable AND/OR operations.
- **Fuzzy neural networks:** Implement logic gates as neural network layers with fuzzy activation functions.
- **Hybrid architectures:** Combine symbolic logic modules with neural networks (e.g., via MathCoder or DeepProbLog) for enhanced reasoning and interpretability.

7. Integration with Diffusion-Based Forward Market Predictors and Institutional Trading Rules

7.1. Diffusion Models for Market Forecasting

Diffusion-based models (e.g., DDPM, DDIM) have shown promise in financial time-series forecasting by modeling the conditional distribution of future returns given historical data and covariates. Integration with the logic engine involves:

- **Conditioning:** Use the output of the predicate graph as part of the conditioning information for the diffusion model.
- **Hierarchical attention:** Employ attention mechanisms to fuse logical, technical, and market features for improved predictive performance.

7.2. Encoding and Evaluating Institutional Trading Rules

Institutional trading rules (e.g., VWAP, volume thresholds, regulatory constraints) can be encoded as additional predicate sets or as constraints in the optimization process. Efficient evaluation requires:

- **Rule templates:** Define parameterized templates for common institutional rules.
 - **Predicate invention:** Use neuro-symbolic systems to invent new predicates or rules based on observed data and expert feedback.
-

8. Fuzzy Logic-Based Money Management and Position Sizing

8.1. Fuzzy Inference Systems for Position Sizing

A fuzzy inference system can dynamically determine position sizes based on input variables such as recent performance, trend probability, and volatility. The process involves:

- **Input fuzzification:** Map raw inputs to fuzzy sets using membership functions.
- **Rule evaluation:** Apply fuzzy rules.
- **Defuzzification:** Convert the fuzzy output to a crisp position size.

Example: Fuzzy Rule Base

If percentage-of-change is high-loss, then lot-size is low-lot-size

If probability-of-trend is high, then lot-size is high-lot-size

If volatility is medium, then lot-size is medium-lot-size

8.2. Integration with Logic Engine and Neural Models

The output of the fuzzy money management system can be used to modulate the predictions of the logic engine and neural models, ensuring that position sizes are consistent with both logical constraints and market conditions.

9. Python Code Patterns and Libraries for Logic Graph Architecture

9.1. Graph Construction and Manipulation

- **NetworkX:** For constructing and manipulating predicate graphs.
 - **reviz:** For visualizing recursive call graphs and logic evaluation trees.
-

9.2. Differentiable Logic and Custom Autograd Functions

- **PyTorch custom autograd:** Implement custom differentiable logic operators.
 - **Fuzzy logic libraries:** Use or extend scikit-fuzzy or implement custom t-norms.
-

9.3. Sparse and Memory-Efficient Representations

- **PyTorch sparse tensors:** Efficient storage of large predicate matrices.
- **TensorFlow sparse tensors:** Equivalent functionality for TensorFlow pipelines.

9.4. Automatic Differentiation Through Logical Operations

By implementing logical operators as differentiable functions, the entire predicate graph becomes part of the computational graph, enabling end-to-end training with gradient-based optimization.

10. Scalability, Parallelism, and Robustness

10.1. Scalability and Parallelism

- **Batch evaluation** using vectorized operations and CUDA kernels.
- **Sparse computation** to minimize unnecessary work.

10.2. Robustness Testing and Verification

Recommended practices include:

- In-sample / out-of-sample testing
- Monte Carlo simulation
- Walk-forward analysis
- Stress testing under extreme or synthetic shocks

11. Benchmark Datasets and Evaluation Metrics

11.1. SPY Options Datasets

Historical SPY options datasets with OHLCV, Greeks, implied volatility, and market microstructure variables.

11.2. Evaluation Metrics

- **Predictive accuracy:** Accuracy, F1-score, MSE

- **Trading performance:** Sharpe ratio, maximum drawdown, profit factor
 - **Constraint satisfaction:** Fraction of predictions satisfying constraints
 - **Robustness:** Stability across regimes and stress tests
-

12. Hybrid Symbolic-Neural Architectures and MathCoder Approaches

12.1. MathCoder and Code-Integrated Reasoning

Key takeaways:

- Interleaving code and logic
 - Chain-of-thought reasoning via execution
 - Hybrid training across language, code, and logic
-

12.2. Graph Neural Networks for Predicate Reasoning

Graph neural networks can model predicate graph structure, aggregate information, and capture higher-order dependencies and symmetries.

13. Implementation Guidance and Best Practices

13.1. Modular Design

- Separation of concerns
 - Extensibility for new predicates and models
-

13.2. Testing and Verification

- Unit tests
 - Integration tests
 - CUDA profiling and memory analysis
-

13.3. Documentation and Visualization

- Graph visualization for debugging
 - Comprehensive code documentation
-

Conclusion

Building a recursive logic engine for SPY options trading that integrates Mamba2, Neural CDEs, and advanced mathematical frameworks is a formidable but tractable challenge. By leveraging combinatorics for efficient predicate exploration, set and group theory for structuring and evaluating nested inequalities, and CUDA-accelerated pipelines for scalable computation, it is possible to construct a system capable of reasoning over highly complex, recursive logical structures.

The recommended approach is to represent recursive inequalities as differentiable predicate graphs, employ combinatorial optimization and group-theoretic reductions to manage complexity, and use modern Python libraries and CUDA custom operators for efficient implementation. Rigorous robustness testing, modular design, and hybrid symbolic-neural architectures will ensure that the system is both powerful and reliable in real-world trading environments.

Key Python Libraries and Tools

- `networkx`: Graph construction and manipulation
- `rcviz`: Recursive call graph visualization
- `torch`, `torch.autograd`: Differentiable logic and custom operators
- `mamba_ssm`, `causal_conv1d`: Mamba2 state-space models
- `scikit-fuzzy`: Fuzzy logic systems
- `pybind11`: Python-CUDA integration
- `pandas`, `numpy`: Data handling and numerical computation

Recommended Reading and References

Combinatorics in AI and ML
Set and Group Theory in Logic Systems
Differentiable Logic and Fuzzy Neural Networks
CUDA-Accelerated Predicate Evaluation
Hybrid Symbolic-Neural Reasoning
Robustness Testing in Algorithmic Trading

Example: Mamba2 Model Initialization (python)

```
1. from mamba_ssm import Mamba2
2. import torch
3.
4. batch, length, dim = 2, 64, 384
5. x = torch.randn(batch, length, dim).to("cuda")
6. model = Mamba2(
7.     d_model=dim,
```

```
8.     d_state=64,
9.     d_conv=4,
10.    expand=2,
11.    headdim=48,
12. ).to("cuda")
13. y = model(x)
14. assert y.shape == x.shape
15.
```

Example: Custom CUDA Kernel for Predicate Evaluation (cpp)

```
1. // CUDA kernel for evaluating chained inequalities
2. __global__ void eval_predicates(const float* features, int* results, int num_predicates, int
num_features) {
3.     int idx = blockIdx.x * blockDim.x + threadIdx.x;
4.     if (idx < num_predicates) {
5.         // Evaluate predicate idx using features
6.         // Store result (1 for True, 0 for False) in results[idx]
7.     }
8. }
9.
```

Example: Building a Predicate Graph with NetworkX (python)

```
1. import networkx as nx
2.
3. G = nx.DiGraph()
4. G.add_node('ineq1', expr='H[3]-H[2] > H[5]-H[3]')
5. G.add_node('ineq2', expr='H[10]-C[8] < ...')
6. G.add_edge('ineq1', 'set1')
7. G.add_edge('ineq2', 'set1')
8. G.add_node('set1', op='AND')
9. G.add_edge('set1', 'super_set1')
10.
```

Example: Differentiable AND Operator (python)

```
1. 1. import torch
2. 2. from torch.autograd import Function
3. 3.
4. 4. class FuzzyAnd(Function):
5. 5.     @staticmethod
6. 6.     def forward(ctx, input1, input2):
7. 7.         ctx.save_for_backward(input1, input2)
8. 8.         return input1 * input2 # Product t-norm
9. 9.
10. 10.    @staticmethod
11. 11.    def backward(ctx, grad_output):
12. 12.        input1, input2 = ctx.saved_tensors
13. 13.        return grad_output * input2, grad_output * input1
14. 14.
15. 15. # Usage
16. 16. a = torch.tensor(0.8, requires_grad=True)
17. 17. b = torch.tensor(0.6, requires_grad=True)
18. 18. out = FuzzyAnd.apply(a, b)
19. 19. out.backward()
20. 20.
21. 21.
```